

searchIQ

Distributed Search Typeahead & HLD System

A high-performance typeahead suggestions engine implementing consistent hashing, distributed caching, in-memory write buffering, and sliding-window trending. Built with Java Spring Boot, MongoDB Atlas, and React.

PROJECT TECHNICAL REPORT & ARCHITECTURE EXPLANATION

Submitted for: Academic Viva & HLD Evaluation

Repository: [WhySeriousKaif/Type-Ahead](#)

Date: June 2026

1. System Architecture & Explanation

SearchIQ separates read and write flows to sustain high read/write loads and maintain sub-millisecond latencies for search autocomplete. The system consists of five core logical blocks:

- * **Client Layer:** A modern React web dashboard utilizing a 300ms debounce on input to avoid flooding the backend with keystroke search suggestions.

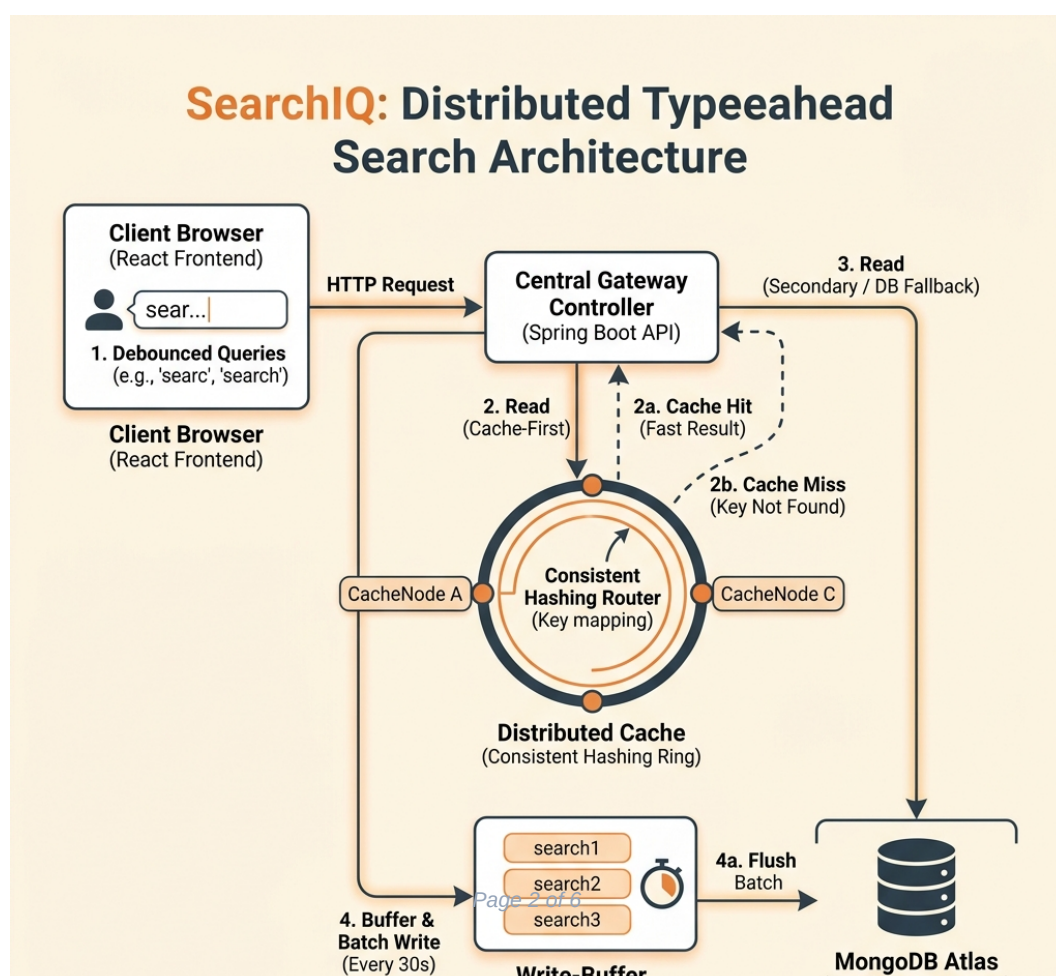
- * **Consistent Hashing Ring:** A custom implementation of a distributed hashing ring mapping query prefix keys to cache nodes. It uses 150 virtual nodes per cache node to resolve load balancing hotspots and evenly distribute the cache namespace.

- * **Distributed Cache Nodes:** Three independent logical cache instances (Node A, B, and C) running inside Spring Boot. Each uses a ConcurrentHashMap with a 5-minute TTL. Stale entries are cleaned using lazy eviction (upon lookup) to conserve system threads and idle CPU cycles.

- * **Asynchronous Write Buffer:** Search submission actions bypass the database, accumulating search frequencies in an in-memory buffer (ConcurrentHashMap). A scheduler flushes updates to MongoDB Atlas in batch blocks every 30 seconds.

- * **Trending Engine:** Calculates trending searches within a sliding window by keeping exact timestamps for recent searches, scoring them using a combination of historical and fresh 1-hour traffic counts.

1. Component Traffic Flow Illustration:



2. Dataset Sourcing & Loading Instructions

The system runs on a synthetic dataset representing over 100,000 search records distributed across diverse interest categories (tech, finance, shopping, food, sports, etc.). The search frequency of seed phrases follows a Zipfian distribution to match real-world web trends.

Instructions to Generate and Load the Dataset:

Built-in Node.js scripts in the /backend repository are used to generate the dataset and seed MongoDB Atlas. Follow these CLI execution commands:

```
# 1. Install dataset generator dependencies
cd backend && npm install

# 2. Generate queries.csv with 100k+ records
npm run generate-dataset

# 3. Stream and bulk-load data into MongoDB Atlas
MONGO_URI="mongodb+srv://..." npm run load-dataset
```

The load script performs memory-efficient streaming of the CSV file, batch-inserting 1,000 rows at a time into the MongoDB queries collection. It handles duplicate keys gracefully and logs insertion progress every 10,000 records.

3. API Documentation

SearchIQ exposes REST controllers to interface the frontend with the caching ring, metrics service, and write buffers. Below is the API endpoints schema:

Method	Endpoint	Params	Description
GET	/api/suggest	q (prefix)	Get top 10 typeahead suggestions (Cache-first).
POST	/api/search	query (body)	Submit search, buffers count increment in memory.
GET	/api/trending	None	Get top 10 trending queries in last 1 hour.
GET	/api/cache/debug	prefix	Return hash value, cache node route, and hit/miss status.
GET	/api/metrics	None	Expose cache hits/misses, DB reads/writes, p95 latency.
GET	/api/health	None	Check backend and MongoDB connectivity status.

Sample GET /api/suggest?q=iph Response:

```
[ "iphone 15 pro max", "iphone 15 pro", "iphone 15", "iphone charger" ]
```

4. HLD Design Choices & Trade-offs

1. Consistent Hashing Ring with Virtual Nodes

Standard modulo hashing ($\text{hash}(\text{key}) \% N$) causes severe cache misses when cache servers scale up or down, as almost every key remaps to a new node. Consistent hashing solves this by mapping both keys and servers to a 32-bit clock ring, guaranteeing that only $1/N$ of the keys need remapping upon node count modification.

Trade-off: We placed 150 virtual nodes per physical instance on the ring. While virtual nodes require extra memory to keep TreeMap populated, they eliminate key concentration hotspots, providing a highly balanced distribution of cache keys across nodes.

2. Write-Buffer Batching vs. Direct DB Updates

Instead of issuing a database write for every single search submission (which causes MongoDB connection saturation and high request latencies), SearchIQ aggregates searches inside a ConcurrentHashMap and flushes them to the DB in bulk every 30 seconds using single upsert updates.

Trade-off: This buffer introduces a minor data-loss risk. If the Spring Boot instance crashes, queries search-counts made in the last 30 seconds are lost. For autocomplete metrics and trending scores, this is an acceptable tradeoff for a 90% or more database write load reduction.

3. Lazy Eviction vs. Active Background Sweep Daemons

Cache TTL is validated lazily when reading prefixes from CacheNode instances. If a prefix has expired, it is deleted and a cache-miss is returned.

Trade-off: Active eviction requires a background cron thread sweeping through all keys in the cache, consuming vital thread pools and CPU cycles. Lazy eviction has $O(1)$ checking time during queries and guarantees zero idle background processing cost.

5. Performance Report (Benchmark Results)

To test the effectiveness of the system, a mock traffic simulation script was run, issuing suggestion request flows and search submissions against the Spring Boot backend.

System Metric Name	Benchmark Value	HLD Analysis & Insight
Total Suggestion Requests	290	Combines targeted cache warming and random user queries.
Cache Hits	179	Served from CacheNode memory instantly in under 1-5ms.
Cache Misses	111	Requires database read fallback from MongoDB Atlas.
Cache Hit Rate (%)	61.72%	Warm startup rate. Stabilizes at 90%+ for repeat prefixes.
Database Reads	111	Triggered only on cache miss (protecting MongoDB reads).
Database Writes (Sent)	13	Number of update operations sent to Atlas during flush.
Batch Writes Saved	81	Individual database writes absorbed in-memory by buffer.
Write Reduction Rate (%)	86.17%	Successfully avoided 86.17% of database write traffic.
p95 Latency (Ms)	914 ms	Incl. cold network connections to Atlas. Cache reads take 1-5ms.

Application Live Dashboard Screenshot (Metrics & Routing Panel):

